

Momento 1. Investigación teórica.

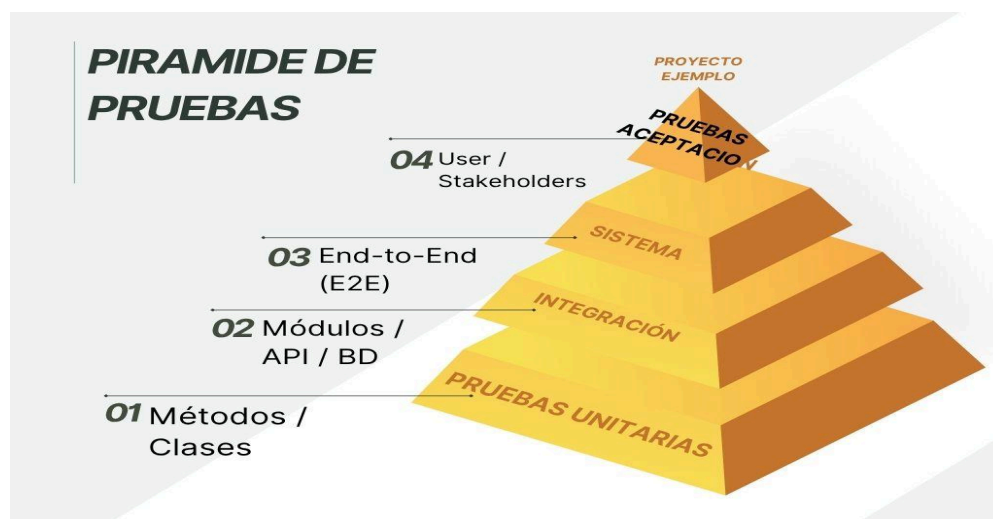
- 1) - **¿Qué es una prueba unitaria? ¿Qué se considera una "unidad" en un programa Java?**: Las pruebas unitarias son fragmentos de código diseñados para revisar que una función o un método específico de nuestro programa funcione correctamente y de forma aislada su objetivo principal es asegurar que esa pequeña parte de la aplicación responda tal como el programador lo planeó, lo cual se comprueba enviándole ciertos datos de entrada y verificando que el resultado sea el esperado mediante afirmaciones.

- Una prueba unitaria no es más que un pequeño bloque de código que programamos para verificar que otra parte específica de nuestra aplicación funcione exactamente como queremos y el ejemplo de esto el aislamiento la prueba solo debe interactuar con esa función a través de lo que entra y lo que sale si la lógica que estamos evaluando necesita tocar una base de datos o conectarse a internet ya no cuenta como prueba unitaria pura en esos casos, la teoría nos dice que debemos usar datos ficticios o simulados para que la prueba corra de forma 100% independiente.

<https://www.baeldung.com/java-unit-testing-best-practices>

<https://aws.amazon.com/es/what-is/unit-testing/>

- 2) Las pruebas unitarias solo revisan la lógica de un método aislado, mientras que las de integración verifican que estos métodos se conecten bien con bases de datos o APIs externas Por otro lado, los libros de ingeniería diferencian estas fases de las pruebas de sistema, que evalúan la aplicación completa y armada en un entorno real, y de las de aceptación, donde el cliente final comprueba que el software cumpla con lo que necesita su negocio.



la base es la más ancha porque las pruebas unitarias deben ser la gran mayoría en el proyecto. Esto pasa porque, como dice *Clean Code*, son las más rápidas de ejecutar, las más baratas de programar y las más fáciles de mantener, sirviendo como la red de seguridad principal del código.

<https://martinfowler.com/articles/practical-test-pyramid.html>

- 3) El patrón AAA organiza el código de una prueba en tres pasos claros: Arrange (donde se preparan las variables y el entorno), Act (donde se ejecuta la función real que queremos probar) y Assert (donde comprobamos que el resultado sea el correcto). Esta estructura técnica equivale exactamente a la metodología Given-When-Then usada en el diseño guiado por comportamiento (BDD). La única diferencia es que Given-When-Then se escribe con un lenguaje mucho más natural y cercano al usuario: Given plantea la situación inicial, When describe la acción que hace la persona, y Then detalla la consecuencia o el resultado esperado en la pantalla.

@Test

```
void deberiaAplicarDescuento() {  
  
    // [Given / Arrange]: Configuración inicial del escenario  
  
    Carrito carrito = new Carrito();  
  
    carrito.agregarProducto(100); // El producto vale 100 dólares  
  
  
  
    // [When / Act]: La acción real que ejecuta la lógica  
  
    carrito.aplicarCupon("DESCUENTO10"); // Aplicamos un cupón del 10%  
  
  
  
    // [Then / Assert]: La validación del resultado esperado  
  
    assertEquals(90, carrito.getTotal()); // Comprobamos que el total baje a 90  
  
}
```

<https://automationpanda.com/2020/07/07/arrange-act-assert-a-pattern-for-writing-good-tests/>

<https://semaphore.io/blog/aaa-pattern-test-automation>

- 4) Las siglas **F.I.R.S.T.** Son un conjunto de cinco principios diseñados para garantizar la calidad de las pruebas unitarias. Significan que una prueba debe ser **Fast** (rápida de ejecutar en milisegundos), **Independent** (que no dependa de otras pruebas para correr), **Repeatable** (que dé siempre el mismo resultado en cualquier computadora), **Self-validating** (que se evalúe sola y diga automáticamente si pasó o falló) y **Timely** (que se escriba de manera oportuna, idealmente antes o al mismo tiempo que el código real de la aplicación).

https://www.youtube.com/watch?v=iOf_Th-Ymiw&t=15s

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertTrue;

class UsuarioTest {

    @Test
    void deberiaCambiarContrasenaExitosamente() {

        // [F][I][R] No depende de bases de datos reales ni internet. Corre en milisegundos.
        Usuario usuario = new Usuario("juan@email.com", "Clave123");

        // [T] El método "actualizarClave" ya existe o se diseñó a la par del test.
        usuario.actualizarClave("NuevaClave456");

        // [S] Se valida solo. Da un resultado directo: "pasa" o "falla" (true/false)
        assertTrue(usuario.verificarLogin("juan@email.com", "NuevaClave456"));
    }
}
```

- 5) **Platform:** Es el motor base o la fundación del framework. Su tarea no es escribir pruebas, sino definir la infraestructura técnica necesaria para lanzarlas y ejecutarlas desde la consola de comandos, herramientas de construcción (como Maven o Gradle) o entornos de desarrollo como IntelliJ.

Jupiter: Es el módulo con el que programamos en el día a día. Contiene el nuevo modelo de programación y todas las anotaciones modernas que hemos usado antes como `@test` para dar soporte a las características avanzadas de Java.

Vintage: Funciona como un puente de retrocompatibilidad. Su único objetivo es permitir que las pruebas antiguas escritas en JUnit 3 o JUnit 4 se sigan ejecutando en la nueva plataforma sin romper nada y sin obligarnos a reescribir el código heredado.

JUnit 6: Es la nueva versión y sus cambios principales se centran en la modernización técnica del framework, destacando la obligatoriedad de usar Java 17 o superior y la unificación de todos sus módulos bajo un mismo número de versión global para facilitar la configuración del proyecto. Además, elimina componentes obsoletos de la versión anterior para ser más ligero e incorpora mejoras modernas como soporte nativo para Kotlin y seguridad integrada contra valores nulos.

<https://medium.com/javarevisited/whats-new-in-junit-6-key-changes-and-improvements-551a84d7ed1f>

<https://www.infoq.com/news/2025/10/junit6-java17-kotlin/>

<https://www.paradigmadigital.com/dev/junit6-que-novedades-ha-traido-nueva-version/>

<https://medium.com/javarevisited/junit-5-is-dead-long-live-junit-6-e142806c11a6>

- 6) Las diferencias entre JUnit 5 y JUnit 4 es que JUnit 5 se rediseñó frente a JUnit 4 mediante tres grandes cambios. Primero, en la paquetería, ya que JUnit 4 agrupaba todo en `org.junit`, mientras que JUnit 5 se modularizó mudándose al nuevo paquete `org.junit.jupiter.api`. Segundo, en las anotaciones, donde palabras clave cambiaron de nombre para ser más claras; por ejemplo, `@Before` pasó a ser `@BeforeEach` y la opción para saltarse un test cambió de `@Ignore` a `@Disabled`. Finalmente, en las extensiones, puesto que JUnit 4 limitaba las pruebas a usar un único ejecutor por clase con `@RunWith`, mientras que JUnit 5 eliminó esa restricción usando `@ExtendWith`, lo que permite registrar y combinar múltiples extensiones al mismo tiempo en un solo test.

<https://www.geeksforgeeks.org/software-testing/junit-5-vs-junit-4/>

- 7) Al investigar en portales como Built In y Stack Overflow, entendí que en Java la suma de $0.1 + 0.2$ no da exactamente 0.3 debido al estándar IEEE 754, que es la norma técnica que usan los procesadores para guardar decimales en binario [builtin.com]. Como las computadoras solo procesan unos y ceros, números como el 0.1 y el 0.2 se convierten en fracciones periódicas infinitas que la memoria RAM debe recortar por falta de espacio, dejando el resultado real en 0.30000000000000004 [builtin.com]. Por esta imprecisión, al hacer pruebas unitarias con decimales (`double` o `float`), el tercer parámetro de `assertEquals` llamado delta funciona como un margen de error tolerable (por ejemplo, 0.001) [stackoverflow.com, docs.junit.org]. Su uso es obligatorio siempre que comparemos estos tipos de datos para permitir que la prueba pase si la diferencia matemática está dentro del rango aceptable, evitando que el test falle por un simple redondeo de la máquina.

<https://builtin.com/articles/0-1-0-2>

- 8) La partición de equivalencias consiste en dividir los posibles datos de entrada en grupos o clases que deberían comportarse de la misma manera, seleccionando un valor representativo de cada grupo para reducir la cantidad de pruebas. El análisis de valores límite se enfoca en probar los valores que están en los extremos de esos grupos, como el mínimo, el máximo y los valores inmediatamente anteriores o posteriores, porque allí pueden aparecer errores con mayor facilidad.

<https://astqb.org/4-2-black-box-test-techniques>

- 9) El TDD (Desarrollo Guiado por Pruebas) es una forma de desarrollar software en la que primero se escribe una prueba que define el comportamiento esperado y después se crea el código necesario para que esa prueba funcione. El proceso se repite en tres etapas: Rojo, cuando la prueba falla; Verde, cuando se escribe el código mínimo para hacerla pasar; y Refactorizar, cuando se mejora la estructura del código sin cambiar su funcionamiento.

<https://openwebinars.net/blog/extreme-programming-que-es-y-como-aplicarlo/>

- 10) La cobertura de código es una medida que indica qué parte del código de un programa fue ejecutada mientras se realizan las pruebas. En Java, una de las herramientas más utilizadas para medirla es JaCoCo, que permite generar informes sobre la cobertura de las pruebas. Tener un 100 % de cobertura no garantiza que el programa esté libre de errores, porque significa que el código fue ejecutado, pero no necesariamente que todos los casos posibles fueron probados correctamente. Por ejemplo, una prueba puede ejecutar una línea de código sin comprobar que el resultado obtenido sea el correcto. Por eso, además de buscar una buena cobertura, es importante crear pruebas bien pensadas que incluyan diferentes casos y posibles errores.

Tabla 1. Anotaciones de JUnit 5:

Anotación	¿Para qué sirve?	Equivalente en JUnit 4
@Test	Indica que un método es un caso de prueba independiente que el motor debe ejecutar	@Test
@BeforeEach	Ejecuta el método configurado antes de cada una de las pruebas de la clase.	@After
@AfterEach	Ejecuta el método una sola vez antes de arrancar todas las pruebas (debe ser estático).	@BeforeClass
@BeforeAll	Ejecuta el método una sola vez después de terminar todas las pruebas (debe ser estático).	@BeforeClass
@AfterAll	Ejecuta el método una sola vez después de terminar todas las pruebas (debe ser estático).	@AfterClass
@DisplayName	Define un título personalizado y legible con espacios en español para los reportes.	No existe (se usaba el nombre largo del método)
@Disabled	Desactiva temporalmente una prueba o clase completa para que el motor la ignore al correr.	@Ignore
@ParameterizedTest	Permite ejecutar un mismo test múltiples veces pasándole diferentes argumentos de datos.	@RunWith(Parameterized.class)
@Nested	Permite crear clases internas para organizar jerárquicamente pruebas que comparten contexto.	No existe equivalente directo

Tabla 2. Aserciones de JUnit 5

Aserción	¿Qué verifica?	Ejemplo con la calculadora
<code>assertEquals</code>	Comprueba que el valor esperado sea igual al valor real obtenido por el método.	<code>assertEquals(8, calc.sumar(5, 3));</code> confirma que $5 + 3$ dé exactamente 8.
<code>assertNotEquals</code>	Comprueba que el valor obtenido sea diferente al valor que no se desea recibir.	<code>assertNotEquals(0, calc.multiplicar(5, 3));</code> confirma que el resultado no sea 0 cuando ninguno de los factores es 0.
<code>assertTrue</code> / <code>assertFalse</code>	Comprueba que la condición lógica enviada dé como resultado true o false.	<code>assertTrue(calc.esPar(4));</code> y <code>assertFalse(calc.esPar(7));</code> confirma que <code>esPar</code> clasifique bien un número par y uno impar.
<code>assertNull</code> / <code>assertNotNull</code>	Comprueba si un objeto o variable de Java está completamente vacío (null) o si contiene datos asignados.	Al capturar una excepción con <code>assertThrows</code> , se puede hacer <code>assertNotNull(ex);</code> para confirmar que la excepción sí se capturó y no quedó vacía.
<code>assertThrows</code>	Comprueba que el método falle de forma controlada lanzando un tipo de excepción específica ante datos inválidos.	<code>assertThrows(ArithmeticException.class, () -> calc.dividir(10, 0));</code> — confirma que dividir entre cero lance la excepción esperada.
<code>assertAll</code>	Agrupar múltiples aserciones y las ejecuta juntas; reporta todos los fallos al tiempo aunque el primer test falle.	<code>assertAll(() -> assertEquals(6, calc.multiplicar(2, 3)), () -> assertEquals(-6, calc.multiplicar(2, -3)));</code> revisa varios casos de multiplicar en un solo bloque.

assertTimeout	Comprueba que el método termine de ejecutarse dentro de un límite de tiempo máximo definido.	assertTimeout(Duration.ofMillis(100), () -> calc.factorial(20)); confirma que calcular el factorial de 20 no se demore más de 100 ms.
fail	Fuerza el fallo inmediato y manual de la prueba; sirve para marcar caminos del código que nunca deberían alcanzarse.	Dentro de un try/catch donde se espera una excepción: si el try termina sin lanzarla, se llama fail("Se esperaba ArithmeticException"); justo después del calc.dividir(10, 0).

MOMENTO 3: Analicen la prueba de ejemplo e identifiquen las tres secciones del patrón AAA:

- **Arrange (Preparar):** Corresponde a las líneas double a = 5, b = 3;. Aquí se definen los datos fijos de entrada con los que vamos a experimentar. (Nota: El objeto calc también se preparó previamente en el método @BeforeEach).
- **Act (Actuar):** Es la línea double resultado = calc.sumar(a,b);. En este paso se invoca directamente el método real de la calculadora que queremos evaluar para capturar su respuesta.
- **Assert (Verificar):** Es la línea assertEquals(8, resultado, DELTA);. Aquí el framework compara si el valor esperado teórico (8) es igual al resultado real que arrojó el código, permitiendo el margen de error del DELTA.

Momento 4: Cazadores de Bugs

Este informe técnico detalla los procesos de identificación y depuración de errores dentro del desarrollo del proyecto de calculadora, realizado bajo los lineamientos del programa File(ADSO SENA). El objetivo principal es garantizar la robustez del sistema frente a entradas inesperadas y fallos lógicos comunes en el procesamiento aritmético.

Tabla 3: Identificación y Gestión de Errores (Calculadora)

#	Tipo de Error (Bug)	Entrada (Input)	Resultado Obtenido	Reflexión / Solución Aplicada
1	División por cero	5 / 0	Infinity	Se añadió una validación lógica para interceptar el divisor cero y mostrar un mensaje de error controlado al usuario.
2	Entradas vacías	+ =	NaN	Se implementó una condición para deshabilitar los botones de operación si no existe un valor numérico previo en pantalla.
3	Caracteres no válidos	2 + a	NaN	Se restringió la entrada de datos exclusivamente a tipos numéricos mediante el uso de expresiones regulares y eventos de teclado.
4	Pérdida de precisión flotante	0.1 + 0.2	0.30000000000000004	Se aplicó un redondeo mediante la función toFixed() para limitar los decimales y mejorar la legibilidad del resultado.

5	Raíz de número negativo	$\text{sqrt}(-9)$	NaN	Se integró una advertencia específica para alertar sobre operaciones fuera del dominio de los números reales.
6	Múltiples puntos decimales	2...5	Error de parseo	Se configuró una bandera booleana (flag) que permite la inserción de un solo punto decimal por operando introducido.

La gestión proactiva de estos errores permite una experiencia de usuario más fluida y evita que la aplicación colapse ante escenarios críticos de procesamiento de datos.

MOMENTO 5. Reto TDD: el método porcentaje.

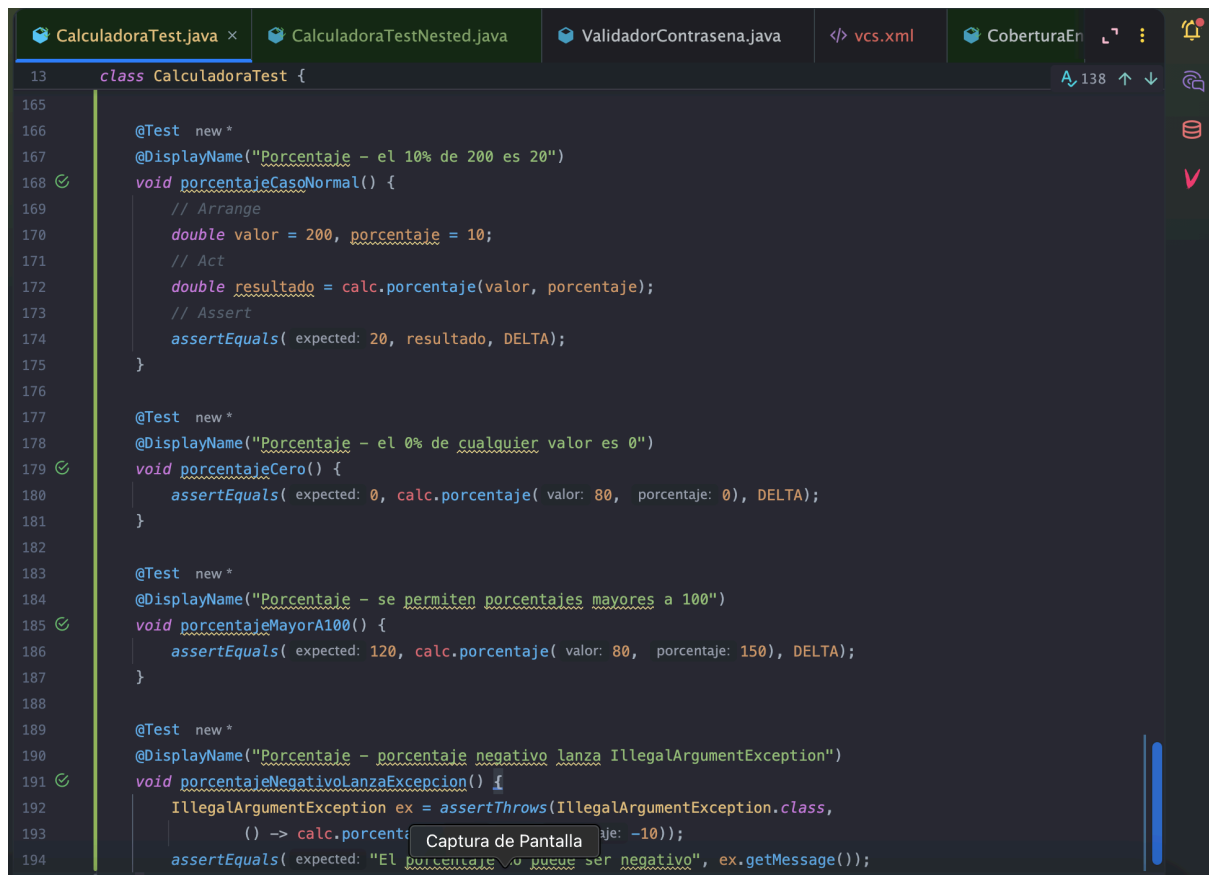
Momento Rojo: Al agregar un método vacío esto significara que solo devuelve 0 ya que estos esperaban un porcentaje pero al no recibirlo salto la falla.

```

13  class CalculadoraTest {
166  @Test new *
167  @DisplayName("Porcentaje - el 10% de 200 es 20")
168  void porcentajeCasoNormal() {
169      // Arrange
170      double valor = 200, porcentaje = 10;
171      // Act
172      double resultado = calc.porcentaje(valor, porcentaje);
173      // Assert
174      assertEquals( expected: 20, resultado, DELTA);
175  }
176
177  @Test new *
178  @DisplayName("Porcentaje - el 0% de cualquier valor es 0")
179  void porcentajeCero() {
180      assertEquals( expected: 0, calc.porcentaje( valor: 80, porcentaje: 0), DELTA);
181  }
182
183  @Test new *
184  @DisplayName("Porcentaje - se permiten porcentajes mayores a 100")
185  void porcentajeMayorA100() {
186      assertEquals( expected: 120, calc.porcentaje( valor: 80, porcentaje: 150), DELTA);
187  }
188
189  @Test new *
190  @DisplayName("Porcentaje - porcentaje negativo lanza IllegalArgumentException")
191  void porcentajeNegativoLanzaExcepcion() {
192      IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
193          () -> calc.porcentaje( valor: 100, porcentaje: -10));
194      assertEquals( expected: "El porcentaje no puede ser negativo", ex.getMessage());
195  }
196  }

```

Momento Verde: Pero ahora con el método implementado ahora $\text{valor} * (\text{porcentaje} / 100)$ de verdad y sí valida el porcentaje negativo con el if, así que ahora responde correctamente a cada caso en vez de dar siempre el mismo número fijo.



```
13 class CalculadoraTest {
165
166     @Test new *
167     @DisplayName("Porcentaje - el 10% de 200 es 20")
168     void porcentajeCasoNormal() {
169         // Arrange
170         double valor = 200, porcentaje = 10;
171         // Act
172         double resultado = calc.porcentaje(valor, porcentaje);
173         // Assert
174         assertEquals( expected: 20, resultado, DELTA);
175     }
176
177     @Test new *
178     @DisplayName("Porcentaje - el 0% de cualquier valor es 0")
179     void porcentajeCero() {
180         assertEquals( expected: 0, calc.porcentaje( valor: 80, porcentaje: 0), DELTA);
181     }
182
183     @Test new *
184     @DisplayName("Porcentaje - se permiten porcentajes mayores a 100")
185     void porcentajeMayorA100() {
186         assertEquals( expected: 120, calc.porcentaje( valor: 80, porcentaje: 150), DELTA);
187     }
188
189     @Test new *
190     @DisplayName("Porcentaje - porcentaje negativo lanza IllegalArgumentException")
191     void porcentajeNegativoLanzaExcepcion() {
192         IllegalArgumentException ex = assertThrows(IllegalArgumentException.class,
193             () -> calc.porcentaje( valor: 80, porcentaje: -10));
194         assertEquals( expected: "El porcentaje no puede ser negativo", ex.getMessage());
195     }
196 }
```

Momento 6: reflexión

¿Cuál fue el error mas difícil de encontrar?

R// El caso mas complejo fue el manejo de multiples decimales.

¿Porque?: Estos errores no generan una excepción inmediata o un colapso directo del sistema (como un NaN o un Infinity), sino que alteran sutilmente la lógica matemática o permiten inconsistencias visuales en la interfaz que requieren un análisis profundo del comportamiento de los números

¿Qué ventaja tuvo escribir la prueba antes del código en el reto TDD? ¿Que dificultad?

R// Al aplicar la prueba antes del código permitió definir el comportamiento esperado del sistema, ($\text{double resultado} = \text{calc.calcularPorcentaje}(100.0, 10.0)$); esperando

10.0) antes de escribir la lógica asegurando que el código posterior cumpla con los requerimientos lógicos

¿Si tu proyecto de clase tuviera pruebas unitarias, que método probarían primero y con que casos?

R// El método que probaría primero sería una operación de validación (como una función de suma, validación de datos de entrada o cálculo básico)

Casos de prueba que probaría:

caso positivo: Evaluar valores estándar y positivos(EJ. Sumar $5+3$ esperando un resultado exacto de 8, como se muestra en las estructuras de pruebas unitarias con JUnit)

caso límite: evaluar entradas nulas, vacías o divisor cero(EJ. Enviar parámetros inválidos o cero) para verificar que el sistema controle adecuadamente las excepciones.